Aprendizado semi-supervisionado e grafos

Emílio Bergamim Júnior

Instituto de Geociências e Ciências Exatas - UNESP

2026

Introdução

- Até aqui, consideramos um cenário em que o conjunto de dados que nos é fornecido é completamente rotulado.
- No caso em que o conjunto é parcialmente rotulado, sempre há a opção de treinar um modelo na parte rotulada e então aplicá-lo no restante.
- Porém, em alguns casos, especialmente para poucos rótulos, esse procedimento pode ser insuficiente e levar a sobreajuste.
- Assim, há de se pensar em como lidar com cenários onde a fração de dados rotulados é muito inferior à de não rotulados.

A dificuldade de rotular

- Obter dados rotulados pode ser muito caro.
- Dados médicos, por exemplo, demandam um especialista para serem propriamente rotulados.
- Em outros casos, pode-se ter uma quantidade muito grande de dados, de forma que a anotação de cada um deles pode demandar muito tempo.
- Isso pode ser ainda identificado como um problema de preencher dados discretos faltantes.

A configuração do problema

- Suponha um conjunto de dados

$$\mathcal{X} = \{\boldsymbol{x}_i\}_{i=1}^{N} \tag{1}$$

e um conjunto de rótulos

$$\mathcal{R} = \{y_i\}_{i=1}^{N_{\mathcal{R}}} \tag{2}$$

com $N_{\mathcal{R}} < N$. De forma que os primeiros $N_{\mathcal{R}}$ sejam rotulados e os demais, não.

Similaridade

- Já discutimos que o sucesso de um modelo não depende apenas deste, mas também dos dados possibilitarem a detecção de algum padrão.
- Sob a hipótese de que há algo a ser detectado, pode-se adotar que as classes em questão estão **estruturalmente relacionadas** isto é: os rótulos não surgiram por acaso, mas descrevem algo estrutural das instâncias.
- **Instâncias próximas possuem rótulos próximos**
- Assim, definimos a similaridade gaussiana entre dois pontos $\boldsymbol{u}$ e $\boldsymbol{v}$:

$$W(\boldsymbol{u}, \boldsymbol{v}) = \exp\left\{-\frac{\|\boldsymbol{u} - \boldsymbol{v}\|^2}{2\sigma^2}\right\} \tag{3}$$

e $\sigma > 0$ é um parâmetro a ser determinado.

Exemplo

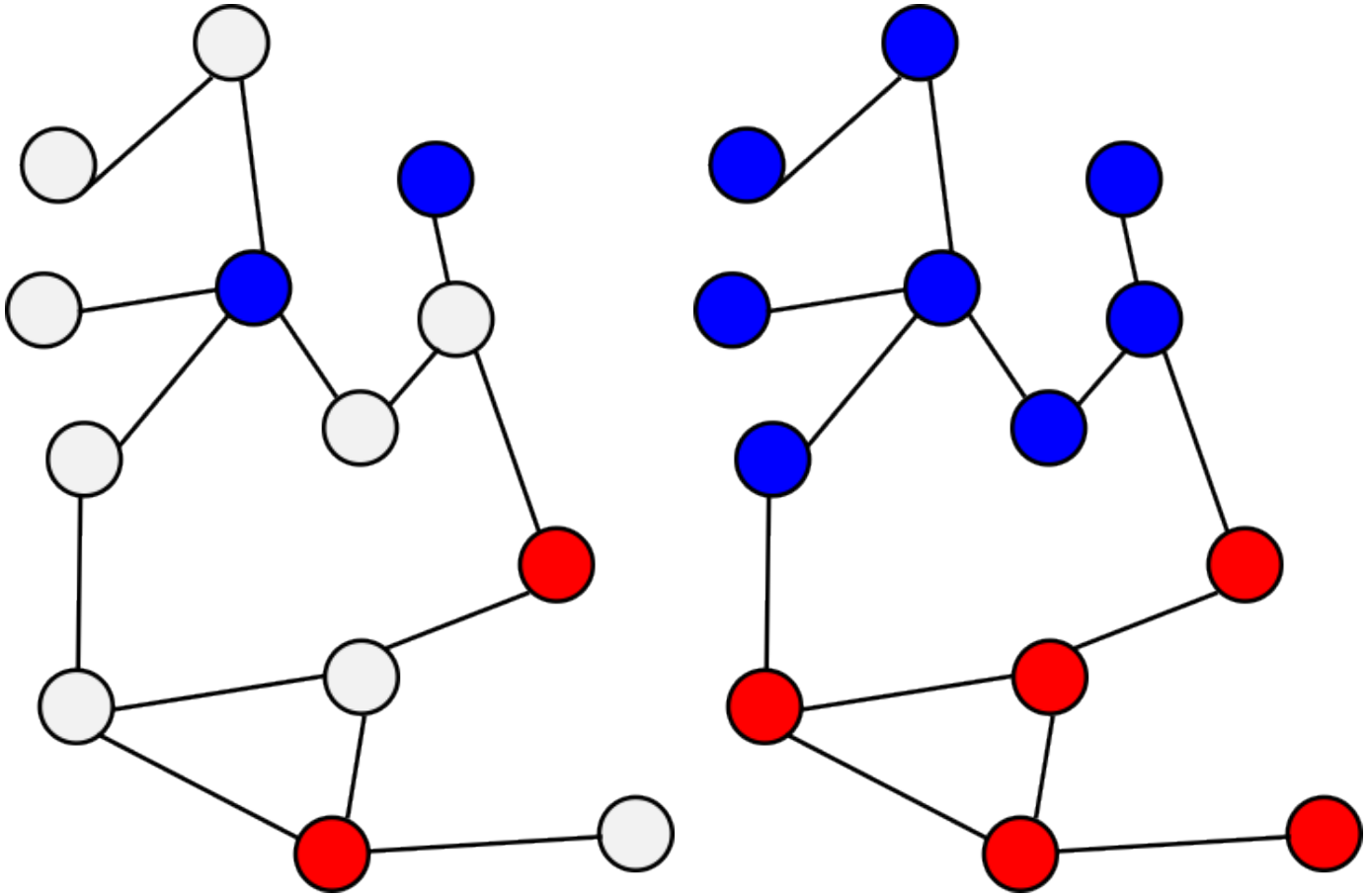


Figura: Exemplo do que queremos fazer. À esquerda os dados como nos são fornecidos. À direita uma possível solução do problema.

Consistência global

- Se existem q rótulos, considere $\{\boldsymbol{\phi}_i\}_{i=1}^{N} \subset \mathbb{R}^q$ a matriz com os rótulos suaves do conjunto (que serão determinados em breve).
- Uma forma de descrever a a condição de que instâncias próximas possuem rótulos próximos é através do custo

$$\mathcal{H}_{global}(\{\boldsymbol{\phi}_i\}) = \sum_{i=1}^{N} \sum_{j=i+1}^{N} W_{i,j} \|\boldsymbol{\phi}_i - \boldsymbol{\phi}_j\|^2 \tag{4}$$

onde

$$W_{i,j} = W(\boldsymbol{x}_i, \boldsymbol{x}_j). \tag{5}$$

- Essa função terá um valor maior conforme os rótulos suaves se distanciam, ao mesmo tempo que diminui conforme os mesmos se aproximam.
- Porém, veja que $\mathcal{H}_{local}$ é minimizada quando todos os rótulos suaves são iguais.

Consistência local

- Ainda não incluimos aquilo que sabemos sobre os rótulos. Considere $\{\boldsymbol{\theta}_i\}_{i=1}^{N} \subset \mathbb{R}^q$

$$\theta_{i,s} = \begin{cases} 1, \text{ se } i < N_{\mathcal{R}} \text{ e } s = y_i \\ 0, \text{caso contrário.} \end{cases} \tag{6}$$

- Defina então

$$\mathcal{H}_{global}(\{\boldsymbol{\phi}_i\}) = \sum_{i=1}^{N} \|\boldsymbol{\phi}_i - \boldsymbol{\theta}_i\|^2 \tag{7}$$

- Nessa função, o valor aumenta conforme viola-se o conhecimento sobre o rótulo original.

Consistência local e global

- Conseguimos agora uma função que descreve o que queremos. Para $\mu > 0$,

$$\mathcal{H}(\{\boldsymbol{\phi}_i\}) = \mathcal{H}_{local}(\{\boldsymbol{\phi}_i\}) + \mu\mathcal{H}_{global}(\{\boldsymbol{\phi}_i\}) \tag{8}$$

- μ controla o balanço entre as duas consistências.
- Agora, o mínimo é não-trivial e fornece um rótulo para cada ponto:

$$\hat{y}_i = arg\max_s \phi_{i,s} \tag{9}$$

- Mas como calcular esse mínimo?
- Vamos começar pelo gradiente, como de costume, mas vamos avaliar o de cada uma das consistências separadamente.

Termo global - Grafos

- Como $\boldsymbol{W}$ é uma matriz simétrica, pode-se identificá-la como a matriz de adjacência ponderada de um grafo.
- Nesse caso, $\mathcal{H}_{global}$ corresponde a uma soma sobre as arestas do grafo.
- A derivada em relação a um rótulo suave corresponde a derivar sobre todas as arestas na qual o rótulo aparece.

$$\frac{\partial \mathcal{H}_{global}}{\partial \phi_{i,s}} = \sum_{j=1}^{N} W_{i,j}(\phi_{i,s} - \phi_{j,s}) \tag{10}$$

- Seja $\boldsymbol{D}$ a matriz diagonal tal que

$$D_{i,i} = \sum_{j=1}^{N} W_{i,j} \quad \text{(Matriz de grau)} \tag{11}$$

Termo global - Grafos

- Veja agora que

$$\frac{\partial \mathcal{H}_{global}}{\partial \phi_{i,s}} = D_{i,i}\phi_{i,s} - \sum_{j=1}^{N} W_{i,j}\phi_{j,s}. \tag{12}$$

- Considerando agora que $\mathbf{\Phi}_{N\times q}$ é a matriz cuja $i-$ésima linha é igual a $\boldsymbol{\phi}_i$, conclui-se que

$$\frac{\partial \mathcal{H}_{global}}{\partial \mathbf{\Phi}} = (\boldsymbol{D} - \boldsymbol{W})\mathbf{\Phi} \tag{13}$$

- E $\boldsymbol{L} = \boldsymbol{D} - \boldsymbol{W}$ é dito o laplaciano de $\boldsymbol{W}$.

Termo local

- Esse termo é mais simples, já que reduz-se à derivada de uma função quadrática.

$$\frac{\partial \mathcal{H}_{local}}{\partial \phi_{i,s}} = \frac{\partial \|\boldsymbol{\phi}_i - \boldsymbol{\theta}_i\|^2}{\partial \phi_{i,s}} = 2(\phi_{i,s} - \theta_{i,s}) \tag{14}$$

- Em forma matricial, isso se resume a

$$\frac{\partial \mathcal{H}_{local}}{\partial \boldsymbol{\Phi}} = 2(\boldsymbol{\Phi} - \boldsymbol{\theta}) \tag{15}$$

Gradiente de $\mathcal{H}$

$$\frac{\partial \mathcal{H}}{\partial \boldsymbol{\Phi}} = 2[(\boldsymbol{D} - \boldsymbol{W})\boldsymbol{\Phi} + \mu(\boldsymbol{\Phi} - \boldsymbol{\theta})] \tag{16}$$

- Igualando essa expressão a zero, conclui-se que

$$\boldsymbol{\Phi} = \mu(\boldsymbol{L} + \mu \boldsymbol{I})^{-1}\boldsymbol{\theta} \tag{17}$$

- Inverter matrizes, no entanto, tem complexidade $O(N^3)$, logo essa via não está disponível para casos com N muito grande.

Algoritmo de propagação

- Podemos chegar na solução iterativamente e com menor custo computacional. Veja essa sequência

$$\boldsymbol{\Phi}_{t+1} = (\boldsymbol{D} + \mu \boldsymbol{I})^{-1}(\boldsymbol{W}\boldsymbol{\Phi}_t + \mu\boldsymbol{\theta}). \tag{18}$$

- No limite $t \to \infty$

$$\boldsymbol{\Phi}_{\infty} = (\boldsymbol{D} + \mu \boldsymbol{I})^{-1}(\boldsymbol{W}\boldsymbol{\Phi}_{\infty} + \mu\boldsymbol{\theta}) \tag{19}$$

$$(\boldsymbol{D} + \mu \boldsymbol{I})\boldsymbol{\Phi}_{\infty} - \boldsymbol{W}\boldsymbol{\Phi}_{\infty} = \mu\boldsymbol{\theta} \tag{20}$$

$$(\boldsymbol{D} - \boldsymbol{W} + \mu \boldsymbol{I})\boldsymbol{\Phi}_{\infty} = \mu\boldsymbol{\theta} \tag{21}$$

$$\boldsymbol{\Phi}_{\infty} = (\boldsymbol{D} - \boldsymbol{W} + \mu \boldsymbol{I})^{-1}\mu\boldsymbol{\theta} = \mu(\boldsymbol{L} + \mu \boldsymbol{I})^{-1}\boldsymbol{\theta} \tag{22}$$

Algoritmo de propagação

- Veja que demonstramos que, quando a sequência converge, ela vai para a solução do nosso problema.
- Ela sempre converge, só não demonstramos isso (use a série geométrica das matrizes para provar).
- Conseguimos sair de um problema $O(N^3)$ para $O(N^2)$.
- Esse algoritmo age propagando os rótulos das instâncias rotuladas para as não rotuladas através da estrutura do grafo.

Implementações eficientes

- Podemos melhorar a complexidade ainda mais.
- Olhando para todas as similaridades, é possível que não precisemos armazenar todas. Na verdade não precisamos nem da maioria.
- Escolha um inteiro k e, para uma linha i de $\boldsymbol{W}$, altere para 0 todas as entradas que não corresponderem aos k vizinhos mais próximos de i (esparsificação KNN).
- Isso resulta em uma matriz esparsa, de forma que podemos armazenar somente as entradas não nulas. Em algumas construções, isso permite que o produto $\boldsymbol{W\Phi}$ seja calculado em tempo $O(Nk)$.
- Uma escolha popular é $k = \log N$.

Ajuste de σ

- Na maioria dos estudos, são feitos testes com diferentes valores de σ.
- Um procedimento de ajuste que não passa por sucessivos teste é usar

$$\sigma = \frac{1}{3N} \sum_{i=1}^{N} \|\boldsymbol{x}_i - \boldsymbol{x}_i^k\|^2, \tag{23}$$

  onde $\boldsymbol{x}_i^k$ é o $k-$ésimo vizinho mais próximo.
- Note também que a construção do grafo costuma considerar a ausência de auto-arestas, implicando que $\boldsymbol{W}$ tem a diagonal nula.
- Uma discussão mais aprofundada está no artigo *Graph construction and b-matching for semi-supervised learning*, por Jebara, Wang e Chang.

Normalização

- Um procedimento comum para garantir estabilidade numérica, dentre outras propriedades.
- **Normalização simétrica** faz a mudança

$$\boldsymbol{W} \leftarrow \boldsymbol{D}^{-1/2}\boldsymbol{W}\boldsymbol{D}^{-1/2} \tag{24}$$

Aprendizado semi-supervisionado

- O método que acabamos de discutir é um exemplo de aprendizado semi-supervisionado.
- Nessa situação, os dados rotulados e não rotulados são processados conjuntamente, diferente do caso supervisionado.
- A abordagem descrita até aqui é transdutiva (se adicionar um novo ponto, precisa rodar o algoritmo novamente).
- Na literatura, é reconhecido que métodos transdutivos semi-supervisionados levam a melhores resultados do que os indutivos, provavelmente em função da ausência de rótulos em cenários extremos.

Propagação de rótulos

- A construção que discutimos até aqui leva à dinâmica de propagação de rótulos que já discutimos.
- No entanto, o algoritmo apresentado é mais utilizado para fins didáticos ou matemáticos.
- Na literatura, você encontrará algoritmos como *Label Propagation* ou *Affinity Propagation*, cujo princípio de funcionamento é o mesmo, mas partindo de uma função custo levemente diferente.
- Existem ainda abordagens que não partem desse princípio, como *Particle Competition and Cooperation* e as *Graph Convolutional Networks*.

Propagação de rótulos

- Veja que, se o grafo pode ser dividido em componentes conexas, e os rótulos descrevem justamente essa divisão, o algoritmo sempre funciona.
- Como o mesmo é uma dinâmica de propagação, as componentes conexas agem como grafos independentes: o rótulo que é propagado em uma não tem nenhuma possibilidade de adentrar a outra.
- Em resumo, o cenário ideal é construir um grafo que tenha essa estrutura para que o algoritmo funcione. A questão é que em geral não é possível garantir isso.
- Porém, é justo pensar que, se a densidade de conexões entre nós de classes distintas for suficientemente pequena, o algoritmo terá sucesso. Por isso o processo de esparsificação é relevante: ele age como uma poda sobre as conexões e, idealmente, irá desfazer a maior parte das arestas equivocadas.

Exercício sugerido

Mostre que, no algoritmo de propagação discutido, se um grafo de similaridade possuir K componentes conexas e os $N_{\mathcal{R}}$ rótulos conhecidos respeitam essa estrutura (isto é, dois pontos rotulados com rótulos diferentes não estão na mesma componente conexa), escolher $\mathbf{\Phi}_0 = \mathbf{0}$ leva a uma solução em que, se dois pontos estão em componentes conexas distintas, então estes possuem rótulos distintos.

Dica: lembre-se que o algoritmo é uma dinâmica de propagação.

O Laplaciano

- O laplaciano do grafo permite detectar várias propriedades do mesmo. Vamos considerar só o termo global do custo discutido anteriormente:

$$\mathcal{H}_{global}(\{\boldsymbol{\phi}_i\}) = \sum_{i=1}^{N} \sum_{j=i+1}^{N} W_{i,j} \|\boldsymbol{\phi}_i - \boldsymbol{\phi}_j\|^2 \tag{25}$$

  e lembramos que a forma matricial do gradiente é

$$\frac{\partial \mathcal{H}_{global}}{\partial \boldsymbol{\Phi}} = \boldsymbol{L}\boldsymbol{\Phi}. \tag{26}$$

- Como essa função é limitada inferiormente, há um mínimo global para a mesma, que será alcançado quando o gradiente for zero.
- Por simplicidade, considere daqui por diante que $\boldsymbol{\Phi}$ tem uma única coluna. Ou seja, é um vetor coluna.

O laplaciano

- Lembre-se que toda matriz está associada a uma transformação linear (e vice-versa).
- Então, para que o gradiente seja nulo em $\mathbf{\Phi}^*$, vamos precisar que

$$\boldsymbol{L}\mathbf{\Phi} = \mathbf{0}. \tag{27}$$

- Veja que essa relação é satisfeita sempre que $\mathbf{\Phi} = c\mathbf{1}$ para algum escalar c, que corresponde ao caso em que todas as entradas de $\mathbf{\Phi}$ são iguais.
- No entanto, ela pode ser satisfeta de outras maneiras?

Núcleo de uma transformação linear

- O núcleo de uma transformação linear corresponde ao conjunto de vetores que levam ao vetor nulo. Em outras palavras, para uma matriz $\boldsymbol{T}$,

$$\text{Nuc}(\boldsymbol{T}) = \{\boldsymbol{u} | \boldsymbol{T}\boldsymbol{u} = \mathbf{0}\}. \tag{28}$$

- Este conjunto é também um espaço vetorial.
- Pelo que já discutimos, sabemos que

$$c\mathbf{1} \in \text{Nuc}(\boldsymbol{L}). \tag{29}$$

Núcleo do laplaciano

- Sabemos que $c\mathbf{1}$ está no núcleo pois

$$\mathcal{H}_{global}(c\mathbf{1}) = 0. \tag{30}$$

- Isso é sempre válido, mas lembrando que $\boldsymbol{W}$ descreve a adjacência de um grafo, note que, se houver mais de uma componente conexa, conseguimos construir outros vetores.

Componentes conexas

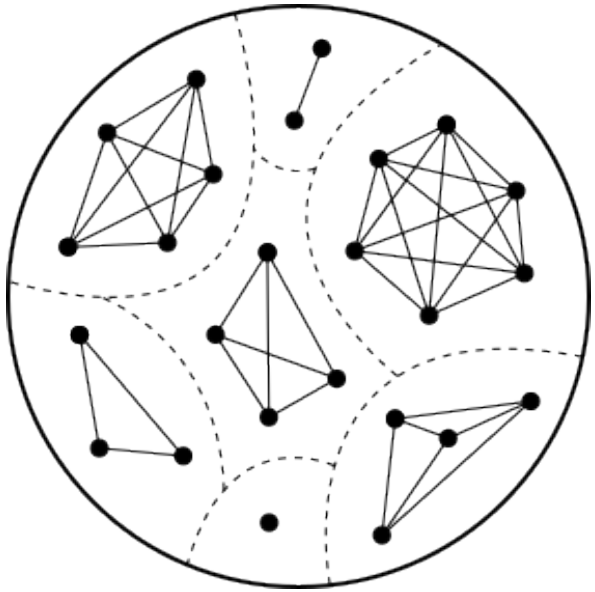


Figura: Exemplo de grafo com múltiplas componentes conexas.

Componentes conexas

- Se existem múltiplas componentes conexas, é possível pensar o grafo como a união de subgrafos conexos.
- Para cada um desses subgrafos, o vetor $c\mathbf{1}$ estará no núcleo do laplaciano desse subgrafo.
- Porém, agora pode-se tomar um valor de c para cada componente conexa. Dessa forma, quando consideramos o grafo original podemos construir um vetor no núcleo que não é da forma $c\mathbf{1}$.

$$\mathbf{\Phi} = (\underbrace{1, 1, 1, 1, 1}_{ComponenteA}, \underbrace{-2, -2, -2, -2}_{ComponenteB}) \tag{31}$$

- Na prática, as componentes não necessariamente aparecem ordenadas dessa forma, mas podemos adotar essa forma nesse argumento sem perda de generalidade.

Componentes conexas e o núcleo do laplaciano

Teorema

O número de componentes conexas em um grafo não-direcionado é igual à dimensão do núcleo do seu laplaciano.

- Esse é apenas o começo de uma série de resultados que permitem conectar álgebra linear e teoria de grafos. A área que estuda esses resultados é chamada Teoria Espectral dos Grafos.
- Nessa direção, um dos resultados mais celebrados é chamado limite de Cheeger, que estabelence que o autovetor associado ao segundo menor autovalor leva a um **corte** quase optimal para dividir um grafo conexo em dois.
- As aplicações disso são muitas, mas aqui vamos nos ater a discutir uma sobre redução de dimensionalidade.

Exercício sugerido

Com base na discussão que fizemos, escreva uma demonstração do teorema anterior.

Laplacian Eigenmaps

- Uma outra propriedade do laplaciano é que seus autovalores sãos todos maiores ou iguais a zero (é uma matriz positiva semi-definida).
- O autovetor zero sempre vai estar associado ao vetor $c\mathbf{1}$, então nessa aplicação ele não será tão interessante.
- Como já discutimos anteriormente, você pode tomar qualquer conjunto de dados e construir um grafo a partir do mesmo. Nesse caso, preferencialmente, um grafo conexo. (Por que?)
- Uma vez que temos esse grafo construído, vamos proceder da mesma maneira que o PCA.

Laplacian Eigenmaps

- Tomamos $\mathcal{H}_{global}$, aplicamos a restrição esférica sobre $\mathbf{\Phi}$ e resolvemos o problema de autovalor

$$\boldsymbol{L}\mathbf{\Phi} = \lambda\mathbf{\Phi}. \tag{32}$$

- Agora, no entanto, podem existir até N autovalores, então seria possível aumentar a dimensionalidade do conjunto, caso seja de interesse.
- Tomando os k primeiros autovetores, formamos uma nova matriz de dados onde cada linha corresponde ao novo conjunto de atributos do instância correspondente.

Laplacian Eigenmaps

- Esse método parte de um pressuposto que o grafo é uma descrição estrutural do conjunto.
- Uma interpretação é que os caminhos nesse grafo formam uma aproximação da distância semântica entre as instâncias.
- A diagonalização do laplaciano age então como uma forma de recriar um conjunto de atributos que descreva as relações entre instâncias de forma mais adequada.

LE versus PCA

- O PCA diagonaliza uma matriz que descreve relações entre atributos, enquanto o LE diagonaliza uma que descreve relações entre instâncias.
- Por conta disso, o grafo pode ser visto como uma reconstrução da variedade que contém os dados.
- Se um conjunto de dados possui d atributos, a complexidade de construir a matriz de covariância é $O(Nd^2)$.
- O que é caro no LE é justamente o procedimento de construção do grafo. Uma vez que determina-se o peso, a esparsificação via K-vizinhos mais próximos tem complexidade $O(dN^2 \log N)$. (Por que?)

Atributos de nós

- Veja, no entanto, que o LE permite aprender atributos a partir de informação estrutural.
- No exemplo discutido, falamos que construir um grafo e diagonalizar o seu laplaciano leva a um novo conjunto de atributos para os dados.
- Porém, seus dados podem já estar na forma de um grafo (a estrutura de uma rede social, por exemplo).
- Assim, construindo o laplaciano e diagonalizando o mesmo pode-se obter um conjunto de atributos que agora será descritivo da estrutura do grafo.
- Em posse de atributos para cada nó, todos os algoritmos já discutidos são agora aplicáveis.

Métodos baseados em grafos

- Métodos em grafos são bastante robustos porque eles essencialmente dependem do grafo possuir as propriedades desejadas.
- Como qualquer conjunto pode ser transformado em um grafo, esta é uma representação útil dos dados.
- O problema é o custo computacional de se construir um grafo, mas existem abordagens com performances mais interessantes do que KNN, como *kd-trees*, entre outras estruturas.